

Algorithmen und Datenstrukturen

Übung 6: Hashing

Robert

2026-06-09

Wiederholung Bitvektor

Bitvektor

- ein Bitvektor ist ein Array von Bits.
- bei der Mengendarstellung steht jedes Bit für ein Element aus dem Universum U
 - jedes Objekt u_i wird durch die Position i im Bitvektor dargestellt

Beispiel

- $U = \langle Hund, Katze, Maus, Floh, Elefant, Mücke \rangle$
- $S_1 = \{Hund, Katze, Maus\} \rightarrow B_1 = (1, 1, 1, 0, 0, 0)$
- $S_2 = \{Hund, Floh, Mücke\} \rightarrow B_2 = (1, 0, 0, 1, 0, 1)$
- $S_3 = \{Maus, Floh, Mücke\} \rightarrow B_3 = (0, 0, 1, 1, 0, 1)$

Aufgabe 1

Sammlung von Informatikbüchern

Nummer	Titel	Autor	Jahr	Kürzel
0	Design Patterns	E. Gamma, et al.	1994	DesP
1	Clean Code	Robert C. Martin	2008	CC
2	Make Your Own Neural Network	Tariq Rashid	2016	MakeNN
3	Agile Software Development	Robert C. Martin	2002	Agile
4	Introduction to Algorithms	T. H. Cormen, et al.	1989	Intro
5	Functional Thinking: Paradigm Over Syntax	Neal Ford	2014	Funct
6	Extreme Programming Explained: Embrace Change	Kent Beck	1999	Extr
7	Algorithms for Reinforcement Learning	Csaba Szepesvari	2010	Algos
8	The Software Craftsman	Robert C. Martin	2014	TheSW
9	Test Driven Development: By Example	Kent Beck	2002	TestDD
10	Programming Pearls	Jon Bentley	1986	PP
11	Building Your Own Compiler with C++	Jim Holmes	1994	Build

1 a)

Gebe die Größe des Universums N an.

- $N = 12$

1 b)

Gebe den Bitvektor an, der das Universum repräsentiert.

- $B(U) = (1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1)^T$

1 c)

Wie sieht der Bitvektor für folgende Menge aus?

$$M_c = \{Agile, TestDD, Build, Intro, TheSW\}$$

Nummer	Titel	Autor	Jahr	Kürzel
0	Design Patterns	E. Gamma, et al.	1994	DesP
1	Clean Code	Robert C. Martin	2008	CC
2	Make Your Own Neural Network	Tariq Rashid	2016	MakeNN
3	Agile Software Development	Robert C. Martin	2002	Agile
4	Introduction to Algorithms	T. H. Cormen, et al.	1989	Intro
5	Functional Thinking: Paradigm Over Syntax	Neal Ford	2014	Funct
6	Extreme Programming Explained: Embrace Change	Kent Beck	1999	Extr
7	Algorithms for Reinforcement Learning	Csaba Szepesvari	2010	Algos
8	The Software Craftsman	Robert C. Martin	2014	TheSW
9	Test Driven Development: By Example	Kent Beck	2002	TestDD
10	Programming Pearls	Jon Bentley	1986	PP
11	Building Your Own Compiler with C++	Jim Holmes	1994	Build

- $B(M_c) = (0, 0, 0, 1, 1, 0, 0, 0, 1, 1, 0, 1)^T$

1 d)

Welche Menge repräsentiert dieser Bitvektor?

$$B(M_d) = (1, 0, 0, 1, 1, 1, 0, 0, 0, 1, 0, 0)^T$$

Nummer	Titel	Autor	Jahr	Kürzel
0	Design Patterns	E. Gamma, et al.	1994	DesP
1	Clean Code	Robert C. Martin	2008	CC
2	Make Your Own Neural Network	Tariq Rashid	2016	MakeNN
3	Agile Software Development	Robert C. Martin	2002	Agile
4	Introduction to Algorithms	T. H. Cormen, et al.	1989	Intro
5	Functional Thinking: Paradigm Over Syntax	Neal Ford	2014	Funct
6	Extreme Programming Explained: Embrace Change	Kent Beck	1999	Extr
7	Algorithms for Reinforcement Learning	Csaba Szepesvari	2010	Algos
8	The Software Craftsman	Robert C. Martin	2014	TheSW
9	Test Driven Development: By Example	Kent Beck	2002	TestDD
10	Programming Pearls	Jon Bentley	1986	PP
11	Building Your Own Compiler with C++	Jim Holmes	1994	Build

- $M_d = \{DesP, Agile, Intro, Funct, TestDD\}$

1 e)

Wie kann ein Algorithmus aussehen, der die Schnittmenge bzw. Vereinigung (M_{result}) zweier Mengen (M_1, M_2) berechnet, die als Bitvektoren dargestellt sind und das Ergebnis auch als Bitvektor darstellt?

1. Gebe den Pseudocode an.
2. Berechne die Laufzeit der Algorithmen.

1. Pseudocode des Algorithmus

- in der Schnittmenge sind die Elemente, die in M_1 und M_2 sind
 - $Bit(M_1 \cap M_2)$
- ein Bitvektor ist ein Array, durch das iteriert werden kann
 - **for**-Schleife

```
1 Algorithmus schnitt(Bit(M_1), Bit(M_2)) :
2     Bit(M_result) = Initialize new Bitvektor
3
4     for i = 0 bis N-1:
5         Bit(M_result)[i] = Bit(M_1)[i] AND Bit(M_2)[i]
6         # bei Vereinigung statt AND ein OR einsetzen
7
8     return Bit(M_result)
```

2. Laufzeit des Algorithmus

```
1 Algorithmus schnitt(Bit(M_1), Bit(M_2)) :  
2     Bit(M_result) = Initialize new Bitvektor  
3  
4     for i = 0 bis N-1:  
5         Bit(M_result)[i] = Bit(M_1)[i] AND Bit(M_2)[i]  
6         # bei Vereinigung statt AND ein OR einsetzen  
7  
8     return Bit(M_result)
```

- Initialisierung des Bitvektors (Zeile 2):
 - $O(N)$, weil alle Werte auf 0 gesetzt werden müssen
- **for**-Schleife (Zeile 4-6):
 - iteriert durch Bitvektoren $\rightarrow O(N)$

- eine Iteration der **for**-Schleife (Zeile 5):
 - Suche nach Elementen = Bit testen $\rightarrow O(1)$
 - logische Operationen (*AND*, *OR*) sind in konstanter Zeit durchführbar $\rightarrow O(1)$
- **return**:
 - $O(1)$ (muss nicht angegeben werden)

$\Rightarrow$ Laufzeit insgesamt:

- $O(N + N \cdot (2 \cdot 1)) = O(N)$

Aufgabe 2

2 a),b)

Gegeben sind Namen von A-Z und eine Haschingfunktion f , die die Namen anhand ihres ersten Buchstaben in ein Array mit 26 verketteten Listen an das Ende der dazugehörigen Liste hängt.

- Vorteile:
 - einfach
 - intuitive
 - einfache Implementation

- Nachteile:
 - Häufung an einzelnen Namen, weil viele Namen mit dem gleichen Buchstaben anfangen
 - wenige Namen fangen mit Q an, viele fangen mit D an
 - möglicherweise lange Suchzeiten bei hohen Füllgraden

Jetzt werden statt verketteten Listen 26 Arrays verwendet. Die Funktion g hängt neue Namen anhand des 2. Buchstaben in das entsprechende Array an.

- Vorteile:
 - schnellere Suchzeit durch mehr Verzweigungen
 - im Array für L kann direkt zwischen Laurin und Linda unterschieden werden, weil anhand des zweiten Buchstaben sortiert wird

- Nachteile:
 - hoher Grundaufwand
 - hohe Kosten bei der Initialisierung ($26 \cdot 26 = 676$ Matrix)
 - wenn nur 100 Namen gespeichert werden, bleibt viel Speicher leer

2 c)

Wäre die Verteilung auf die einzelnen Felder besser, wenn statt des 2. Buchstaben der 3. Buchstabe zur Einordnung in die Arrays genommen würde?

- könnte eventuell besser sein, da phonetische Zusammenhänge umgangen werden (Da- kommt im Deutschen oft vor: Daniel, David)

2 d)

Alternative Vorgehensweisen:

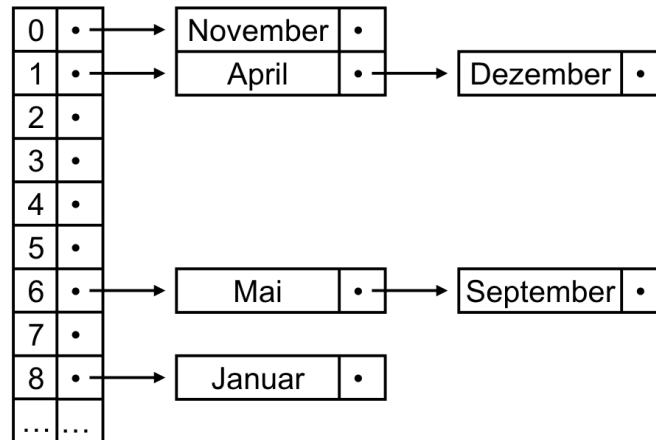
- Baumstruktur
 - von jedem Knoten können 26 Zweige abgehen
 - Vorteil: Speichereffizient und schnelle Suche
 - Nachteil: hoher Implementierungsaufwand

- Buchstabensumme
 - jeder Buchstabe wird in eine Zahl umgewandelt und sie werden aufsummiert
 - jeder Name hat einen eigenen Wert
 - Vorteil: einfach zu berechnen
 - Nachteil: verschiedene Kombinationen können zur gleichen Summe führen (Kollisionen)

Wiederholung offenes Hashing

offenes Hashing

- die Schlüssel werden außerhalb der Tabelle gespeichert
 - z.B. verkettete Liste
- bei Kollisionen werden die Elemente unter der selben Adresse gespeichert



Aufgabe 3

3 a)

Gegeben: $h(k) = k \bmod 7$

Füge die Schlüssel in eine Hashtabelle ein.
Verwende offenes Hashing.

1, 7, 13, 15, 14, 5

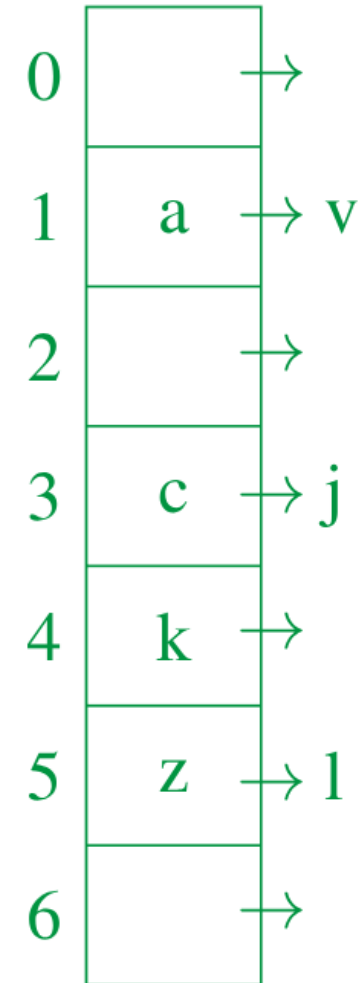
0	7	→ 14
1	1	→ 15
2		→
3		→
4		→
5	5	→
6	13	→

3 b)

Wie können die Buchstaben eingefügt werden?

c, z, a, j, v, k, l

- Die Buchstaben müssen in Zahlen umgewandelt werden:
 - ORD-Funktion
 - ASCII-Kodierung
- mit der ORD-Funktion:
- $c = 3, z = 26, a = 1, j = 10, v = 22, k = 11, l = 12$



Wiederholung geschlossenes/ doppeltes Hashing

geschlossenes Hashing

- bei Kollisionen werden die Elemente mit Hilfe eines Sondierungsverfahren in freie Adressen gespeichert
- jede Adresse nimmt höchstens einen Schlüssel auf
- das Sondierungsverfahren bestimmt die Effizienz

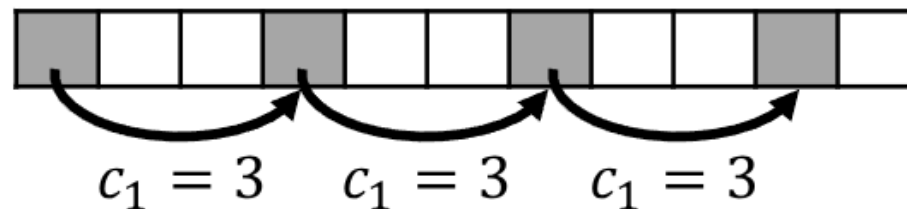
0	November
1	April
2	
3	Dezember
4	
5	
6	Mai
7	September
8	Januar
...	...

lineares Sondieren

- beim linearen Sondieren bestimmt c_1 die Schrittweite, mit der zur nächsten Adresse gesprungen wird

$$h(x, j) = (h(x) + c_1 j) \bmod m$$

- x ist das Element
- j wächst bei jeder Kollision von 0 bis m
- c_1 ist die Schrittweite
- $\bmod m$ sorgt dafür, dass der Index immer im Array bleibt



Doppelhashing

- Doppelhashing verhindert Clusterbildung, indem zwei unabhängige Hashfunktionen (h, h') verwendet werden, wenn es zur Kollision kommt
 - bei der linearen Sondierung wird meist in 1er-Schritten überprüft, ob die nächste Adresse frei ist
 - wenn neue Elemente immer im selben Bereich hinzugefügt werden, kann sich so ein Block bilden, der dann immer weiter wächst
 - Doppelhashing verhindert das, indem die Schrittweite wegen der beiden unabhängigen Hashfunktionen vom Element selber beeinflusst wird
- mit der Hashfunktion $h(x)$ wird die Position berechnet
- bei einer Kollision wird mit $h(x, j) = (h(x) + h'(x) \cdot j) \bmod m$ sondiert
 - $h(x)$ ist die normale Hashfunktion
 - $h'(x)$ ist die zweite unabhängige Hashfunktion
 - j wächst mit jeder Kollision um 1 (nach der ersten Kollision ist $j = 1$)

Aufgabe 4

4 a)

Gegeben: 13, 7, 31, 19, 27, 42, 69, 96

Benutze geschlossenes Hashing mit $h(k) = k \bmod 11$. Kennzeichnen Kollisionen und löse sie durch lineares Sondieren mit $c_1 = 1$. Gebe an, wie oft jeder Schlüssel sondiert wurde.

- $13 \rightarrow 2$
- $7 \rightarrow 7$
- $31 \rightarrow 9$
- $19 \rightarrow 8$
- $27 \rightarrow 5$
- $42 \rightarrow 9 \rightarrow 10$
- $69 \rightarrow 3$
- $96 \rightarrow 8 \rightarrow 9 \rightarrow 10 \rightarrow 0$

4 b)

Benutze Doppel-Hashing mit der Hashfunktion $h(k) = k \bmod 11$, der sekundären Hashfunktion $h'(k) = k \bmod 9 + 1$ und der Sondierungsfunktion $h_j(k) = (h(k) + j \cdot h'(k)) \bmod 11$

- $13 \rightarrow 2$
- $7 \rightarrow 7$
- $31 \rightarrow 9$
- $19 \rightarrow 8$
- $27 \rightarrow 5$

- $42 \rightarrow 9$ (Kollision)
 - $h_1(42) = (h(42) + 1 \cdot h'(42)) \bmod 11 = 5$ (Kollision)
 - $h_2(42) = (h(42) + 2 \cdot h'(42)) \bmod 11 = 1$
- $69 \rightarrow 3$
- $96 \rightarrow 8$ (Kollision)
 - $h_1(96) = (h(96) + 1 \cdot h'(96)) \bmod 11 = 4$